编号 2026160628 研究类型 应用研究 分类号 TP391.1

湖北師範大學
HUBEI NORMAL UNIVERSITY

学士学位论文（设计）

Bachelor's Thesis

论 文 题 目	基于 LangChain 的智能客服平台的设计与实现
作 者 姓 名	司洪宇
学 号	2022116060628
所 在 院 系	人工智能与计算机学院
学科专业名称	人工智能
导 师 及 职 称	孙玉霞 副教授
论文答辩时间	2026 年 4 月 19 日

学士学位论文（设计）诚信承诺书

学 生 承 诺

我承诺在学士学位论文（设计）活动中遵守学校有关规定，恪守学术规范，本人学士学位论文（设计）内容除特别注明和引用外，均为本人观点，不存在剽窃、抄袭他人学术成果，伪造、篡改实验数据的情况。如有违规行为，我愿承担一切责任，接受学校的处理。

学生（签名）：

年　　月　　日

指导教师承诺

我承诺在指导学生学士学位论文（设计）活动中遵守学校有关规定，恪守学术道德规范，经过本人核查，该生学士学位论文（设计）内容除特别注明和引用外，均为该生本人观点，不存在剽窃、抄袭他人学术成果，伪造、篡改实验数据的现象。

指导教师（签名）：

年　　月　　日

目　录

1 绪论.....1
1.1 研究背景与意义.....1
1.2 国内外研究现状.....2
2 相关技术介绍.....3
2.1 大语言模型（LLM）.....3
2.2 LangChain 框架.....4
2.3 数据库.....6
2.4 Spring Boot+Vue.....8
3 系统需求分析.....10
3.1 系统概述.....10
3.2 功能需求分析.....10
3.3 非功能需求分析.....13
4 系统设计.....14
4.1 系统架构设计.....14
4.2 系统功能模块设计.....15
4.3 数据库设计.....22
5 系统功能实现.....27
5.1 用户模块.....27
5.2 商品模块.....30
5.3 智能客服模块.....32
6 总结与展望.....35
6.1 全文总结.....35
6.2 不足与展望.....36
参考文献.....38

基于 LangChain 的智能客服平台的设计与实现

司洪宇（指导教师，孙玉霞 副教授）

（湖北师范大学人工智能与计算机学院 中国 黄石 435002）

摘　要：随着大语言模型技术的快速发展，基于人工智能的智能交互方式逐渐在电子商务领域得到广泛应用。针对传统电商系统交互方式单一、用户操作复杂以及智能化程度不足等问题，本文设计并实现了一种集成基于LangChain的AI客服模块的电商购物系统。系统在传统电商平台基础上，引入大语言模型与工具调用机制，实现自然语言驱动的业务操作。在系统实现方面，基于LangChain框架构建智能客服模块，通过Agent机制将模型能力与业务功能相结合，设计并实现了多种工具（Tools），包括商品购买、订单取消、商品销量查询、库存查询及商品推荐等功能，使系统能够根据用户输入自动完成相应业务操作，从而实现由“问答型客服”向“任务型智能助手”的转变。在系统架构方面，采用SpringBoot与Vue构建前后端分离的电商平台，为大语言模型在电商系统中的实际应用提供了一种可行方案，对智能电商系统的设计与实现具有一定参考价值。

关键词： LangChain；大语言模型；智能客服；SpringBoot；Vue

中图分类号：TP391.1

Design and Implementation of a LangChain-Based Intelligent Customer Service System

SI Hongyu (Tutor: SUN Yuxia)

(School of Artificial Intelligence and Computer Science, Hubei Normal University, Huangshi,China,435002)

Abstract: With the rapid development of large language model (LLM) technology, intelligent interaction methods based on artificial intelligence are increasingly being applied in the field of e-commerce. To address the issues of single interaction mode, complex user operations, and insufficient intelligence in traditional e-commerce systems, this paper designs and implements an e-commerce shopping system integrated with an AI customer service module based on LangChain.Built upon a traditional e-commerce platform, the system introduces large language models and a tool invocation mechanism to enable natural language-driven business operations. In terms of system implementation, the intelligent customer service module is developed based on the LangChain framework. Through the Agent mechanism, model capabilities are integrated with business functions, and multiple tools are designed and implemented, including product purchasing, order cancellation, sales statistics querying, inventory querying, and product recommendation. This allows the system to automatically complete corresponding business operations according to user input, thereby transforming from a question-answering customer service system into a task-oriented intelligent assistant.In terms of system architecture, a front-end and back-end separation architecture based on SpringBoot and Vue is adopted. This study provides a feasible solution for the practical application of large language models in e-commerce systems and offers valuable reference for the design and implementation of intelligent e-commerce platforms.

Keywords: LangChain; Large Language Models; Intelligent Customer Service; SpringBoot; Vue

基于 LangChain 的智能客服平台的设计与实现

司洪宇（指导教师，孙玉霞 副教授）
（湖北师范大学计算机与信息工程学院 中国 黄石 435002）

1 绪论

1.1 研究背景与意义

随着人工智能技术的不断发展，大语言模型在自然语言处理领域展现出强大的语义理解与文本生成能力，推动了智能问答系统从传统基于规则与模板的方法向基于语义理解的智能化方向转变[1]。在这一过程中，LangChain 作为面向大模型应用开发的重要框架，通过对模型调用流程、外部工具集成以及上下文管理机制的封装，使复杂应用系统的构建更加高效与模块化[2]。基于该框架构建的问答系统已经在多个领域得到验证，相关研究表明，通过面向特定领域构建知识问答系统，可以有效提升信息检索与生成的结合能力，从而改善用户交互体验[3]。

随着研究的深入，单纯依赖大模型生成内容已难以满足高准确性与高可靠性的应用需求，引入外部知识库成为重要发展方向。已有研究通过检索增强生成方法，将知识检索与文本生成过程相结合，从而显著提升了问答系统的准确性与可解释性[4]。与此同时，将知识图谱引入问答系统，使模型能够利用结构化知识进行推理，也在一定程度上提升了系统对复杂问题的处理能力[5]。此外，通过外挂专业知识库的方式扩展模型知识边界，使其能够更好地适应垂直领域需求，这一方法已在智能制造等场景中得到验证[6]。

在技术发展之外，应用需求的增长也为智能客服系统的发展提供了重要驱动力。随着电商平台规模的不断扩大，用户服务需求日益增长，人工客服模式面临成本高、效率低等问题。相关研究指出，人工智能技术正逐步在电商平台中落地应用，其中智能客服已成为重要的发展方向之一[7]。同时，推荐系统的发展表明，基于用户行为数据实现个性化服务已成为提升用户体验的关键手段，这为智能客服系统的智能化发展提供了有益参考[8]。在此基础上，引入情感分析技术，可以进一步提升系统对用户意

图与情绪的识别能力，从而实现更加自然和精准的人机交互[9]。

此外，随着多模态技术的发展，信息处理已不再局限于单一文本形式。通过对多模态数据进行语义分析，可以提升系统对复杂信息的理解能力，为智能客服系统处理图文等多源数据提供支持[10]。在具体应用层面，基于 LangChain 构建的行业问答系统以及智能体系统不断涌现，相关研究表明，该类系统在专业领域与教育场景中具有良好的应用效果与扩展能力[11]。这些成果为构建通用化智能客服平台提供了重要的技术基础与实现路径。

综上所述，在大语言模型与 LangChain 框架快速发展的背景下，结合知识增强、多模态处理与智能交互等关键技术，设计并实现一个高效、可扩展的智能客服平台，不仅有助于提升企业服务效率与用户体验，也能够推动大模型技术在实际业务场景中的落地应用[12]，具有重要的理论意义与现实价值。

1.2 国内外研究现状

随着大语言模型技术的发展，智能问答系统逐渐成为人工智能领域的重要研究方向。早期研究多集中于基于框架的系统设计与实现，通过构建完整的问答流程，为后续应用奠定基础。例如，有研究基于 LangChain-LLMs 框架实现了智能问答系统的整体架构设计，涵盖了问题理解、知识检索与答案生成等关键环节，体现了大模型应用的基本实现路径[1]。在此基础上，国外研究进一步从工程化角度对 LangChain 应用开发进行了系统总结，提出了包括链式调用、工具集成以及智能体构建在内的多种方法，为复杂应用系统的开发提供了通用解决方法。

随着技术的不断演进，国内研究逐渐从基础架构转向具体应用场景的实现。相关研究表明，基于 LangChain 构建面向特定领域的知识问答系统，可以有效整合领域知识资源，提高问答系统的专业性与实用性[3]。同时，检索增强生成技术被广泛引入，通过将外部知识库与生成模型结合，显著提升了系统在复杂问题场景下的准确性与稳定性。此外，将知识图谱融入问答系统，使模型具备一定的结构化推理能力，也成为当前研究的重要方向之一。

在知识增强方面，研究者进一步探索通过外挂专业知识库扩展模型能力的方法，使大语言模型能够更好地适应垂直领域应用需求，这一思路在智能制造等场景中已得到初步验证[6]。与此同时，随着互联网行业的发展，人工智能技术在电商平台中的应用不断深化，其中智能客服被认为是重要发展方向之一，这为相关系统研究提供了明

确的应用背景。推荐系统作为另一类典型应用，通过分析用户行为实现个性化服务，其设计思想也为智能客服系统优化用户体验提供了参考。

在国外相关研究中，情感分析与预测模型的结合应用较为成熟，通过对用户文本情感的分析，可以辅助系统更准确地理解用户需求，从而提升交互效果。此外，多模态语义分析逐渐成为研究热点，通过对文本、图像等多源数据进行融合处理，可以增强系统对复杂信息的理解能力，为智能客服系统拓展应用场景提供技术支持[10]。

从具体应用实践来看，基于 LangChain 与大模型的系统已逐步向行业化与智能体方向发展[6]。例如，在专业领域中，通过融合多种模型与工具构建问答系统，能够满足特定行业的知识服务需求；在教育领域中，基于大模型的智能体系统通过任务驱动与对话交互相结合，展现出良好的辅助能力与扩展潜力。这些研究成果表明，智能问答系统正逐步由单一功能模块向综合性智能服务平台演进[4]。

总体来看，国内外研究在智能问答系统领域已取得一定进展，形成了以 LangChain 为核心的应用开发模式，并在知识增强、多模态处理及行业应用等方面不断深化。然而，现有研究仍多集中于单一场景或功能实现，在系统通用性、跨领域知识融合以及复杂业务适配能力等方面仍存在一定不足。因此，有必要在现有研究基础上，进一步探索面向实际应用需求的智能客服平台设计与实现方法[1]。

2 相关技术介绍

2.1 大语言模型（LLM）

大语言模型（Large Language Model, LLM）是近年来自然语言处理领域的重要研究方向，其通过在大规模语料数据上进行训练，学习语言的语义表示与上下文关系，从而具备文本理解与生成能力。目前，大语言模型已成为智能问答系统与智能客服平台的核心技术基础，并在实际应用中展现出良好的效果。相关研究表明，基于 LangChain 等框架构建的大模型应用，可以实现对模型能力的有效组织与调用，从而支撑复杂对话系统的实现[3]。

（1）基本原理

大语言模型通常基于 Transformer 架构构建，该架构通过自注意力机制实现对输入序列中各元素之间关系的建模。相比传统的循环神经网络结构，Transformer 能够并

行处理文本信息，在长文本建模中具有更高的效率和更强的表达能力。在具体实现中，模型通过将输入映射为查询（Query）、键（Key）和值（Value）向量，计算其相似度以获取注意力权重，从而捕捉上下文中的关键信息。这种机制使模型能够更准确地理解复杂语义关系，在问答系统中表现出较强的语义理解能力[4]。此外，当前主流生成式模型多采用自回归方式逐步生成文本，使输出结果在语义与逻辑上保持连贯性，这一特性已在多种问答系统中得到验证。

（2）训练机制

大语言模型的训练通常分为预训练和微调两个阶段。在预训练阶段，模型利用大规模无标注文本数据（如网页、书籍和新闻等）进行学习，通过语言建模或掩码语言建模等任务掌握基本的语法结构和语义关系，从而获得通用语言表示能力。

在具体应用中，模型还需要针对特定任务进行优化，即微调或指令对齐。传统方法是基于标注数据进行微调，而近年来更常采用指令微调（Instruction Tuning）和基于人类反馈的强化学习（RLHF）等方法。这些技术能够使模型生成的内容更加符合人类表达习惯，并在一定程度上提高系统的安全性和可控性[5]。

（3）上下文建模与多轮对话能力

大语言模型具备较强的上下文建模能力，能够支持多轮对话场景。在实际应用中，模型通常通过整合历史对话信息来理解当前用户意图，从而实现连续对话与语义关联。这一能力在智能问答系统中已得到广泛应用，并能够有效提升用户交互体验。同时，结合记忆机制与外部知识存储，可以进一步增强系统在长对话场景中的表现能力。此外，在行业应用中，通过构建面向特定领域的问答系统，可以使模型更准确地理解上下文语义，从而提升服务质量[11]。

（4）优势与特点

在智能客服系统中，大语言模型相比传统基于规则或检索的方法具有显著优势。首先，其具备较强的语义理解能力，能够处理复杂、多样化的自然语言表达；其次，模型具有良好的生成能力，可以输出自然流畅且符合语境的回答；同时，其泛化能力较强，无需针对每一种问题进行单独建模；此外，大语言模型还具备多任务处理能力，能够同时支持问答、翻译、文本摘要等多种功能[6]。

2.2 LangChain 框架

LangChain 是一个面向大语言模型（LLM）应用开发的开源框架，旨在简化基于

大模型的应用构建过程。该框架通过对模型调用流程进行模块化封装，使开发者能够方便地将语言模型与外部数据源、工具接口以及业务逻辑进行整合。在智能客服系统中，LangChain 可以有效实现对话管理、知识检索以及复杂任务处理等功能[1]。

（1）框架概述与核心思想

随着大语言模型能力的不断提升，单纯调用模型接口已经难以满足复杂应用需求，例如多轮对话管理、知识库检索、外部 API 调用以及复杂逻辑控制等问题。LangChain 通过“链式调用”的思想，将多个功能模块进行组合，形成完整的处理流程，从而提升系统的可扩展性和灵活性。其核心可以概括为：将大语言模型与数据、工具以及控制逻辑进行有机连接，从而构建结构化的智能应用系统[3]。

（2）核心组件构成

LangChain 框架由多个核心组件构成，这些组件可以根据实际需求灵活组合，主要包括 Chains、Agents、Tools 和 Memory。Chains 是最基础的组成部分，用于将输入处理、Prompt 构建、模型调用以及输出解析等步骤按顺序连接起来，形成稳定的执行流程；Agents 则在此基础上引入动态决策能力，可以根据用户输入自主选择处理方式，并决定是否调用外部工具；Tools 作为具体功能实现载体，提供数据库查询、API 调用或信息检索等能力，使系统能够执行实际业务操作；Memory 则用于保存对话历史，使系统能够在多轮对话中理解上下文，从而保证交互的连贯性和准确性[3]。各组件之间相互协作，使系统既具备流程化控制能力，又具有良好的灵活性。

（3）工作机制与执行流程

LangChain 在实际运行中遵循一定的处理流程。系统首先接收用户输入，并结合历史对话信息进行预处理，然后构造符合任务需求的 Prompt。在此基础上，根据业务逻辑选择调用 Chain 或 Agent 进行处理；如果任务涉及知识查询，则会触发检索模块获取相关内容，并将其与用户问题一起输入大语言模型进行推理。最终，模型生成回答并返回用户，同时更新对话记忆[11]。在这一过程中，Chain 负责流程组织，Agent 负责策略决策，Tools 提供外部能力支持，Memory 保障上下文连续性，从而共同完成完整的问答任务。

（4）与 RAG 及外部系统的集成能力

LangChain 具有良好的扩展能力，可以方便地与检索增强生成（RAG）技术以及外部系统进行集成。在典型应用中，系统首先将用户问题转换为向量表示，并在向量

数据库中检索相关文档，然后将检索结果作为上下文与问题一起输入模型，从而生成更加准确的回答。LangChain 提供了文档加载、文本切分、向量存储和检索器等组件，使开发者能够快速构建基于知识库的问答系统。同时，通过 Tools 机制，还可以接入数据库、搜索引擎或企业内部接口，实现订单查询等业务功能，从而显著提升系统实用性[6]。

（5）在智能客服系统中的应用与优势

在智能客服系统中，LangChain 具有明显优势。其模块化设计使系统结构清晰，便于后期维护与功能扩展；通过 Memory 机制可以有效支持多轮对话，提高用户交互体验；结合 RAG 技术后，可以利用外部知识库提升回答准确性，减少模型生成错误信息的情况。在本文设计的系统中，LangChain 主要用于组织问答流程、管理对话上下文以及集成知识检索功能，整体流程包括用户输入、上下文获取、知识检索、Prompt 构建以及模型生成回答等步骤。通过该框架的引入，系统能够实现灵活、高效且可靠的智能问答服务。

2.3 数据库

数据库技术是信息系统中用于数据存储、管理与访问的核心技术，在智能客服系统中主要用于支撑用户数据、业务数据以及知识数据的高效管理。通过数据库系统，可以实现数据的结构化存储、快速查询以及安全控制，从而保障系统的稳定运行和数据的一致性。相比简单的文件存储方式，数据库具有更高的可靠性、扩展性和查询效率，是构建企业级应用系统的重要基础[14]。

（1）基本原理

数据库技术的核心在于通过数据模型对现实世界的信息进行抽象，并以结构化形式存储在数据库中。常见的数据模型包括关系模型、文档模型等，其中关系型数据库通过表（Table）的形式组织数据，并通过主键、外键等约束维护数据之间的关系。

在用户访问系统时，应用程序通过数据库管理系统（DBMS）执行查询操作，从数据库中获取所需数据，并进行处理后返回结果。数据库管理系统负责数据的存储、索引、查询优化以及事务管理等功能，使数据访问更加高效和可靠[14]。

（2）关键技术组成

数据库系统通常由多个关键技术模块组成，包括数据建模、索引机制以及查询优化等。

在数据建模阶段，需要根据业务需求设计数据库结构，包括数据表的划分、字段定义以及表之间的关系。这一过程直接影响系统的数据组织方式和查询效率。

索引技术用于提高数据查询速度。通过为关键字段建立索引，可以显著减少查询时的数据扫描范围，从而提升系统响应性能。常见索引结构包括 B+树索引和哈希索引等。

查询优化技术通过分析 SQL 语句执行计划，选择最优的查询路径，从而提高数据检索效率。此外，事务管理机制能够保证数据操作的原子性、一致性、隔离性和持久性（ACID），确保系统在高并发场景下的数据可靠性。

（3）工作流程

在实际应用中，数据库的工作流程可以概括为以下几个步骤：首先，系统接收用户请求，并将其转换为数据库查询语句；其次，数据库管理系统对查询语句进行解析与优化，生成执行计划；然后，根据执行计划从数据表或索引中检索相关数据；最后，将查询结果返回应用层，并展示给用户。

在这一过程中，数据库负责提供高效、可靠的数据支持，而应用层负责业务逻辑处理和结果呈现，两者相互配合，共同完成系统功能。这种流程保证了数据访问的高效性和系统运行的稳定性[14]。

（4）优势与应用价值

相比简单的数据存储方式，数据库技术具有显著优势。首先，它能够实现数据的结构化管理，提高数据的一致性与完整性；其次，通过索引和查询优化机制，可以大幅提升数据检索效率；此外，数据库支持并发控制和事务管理，使系统能够在多用户环境下稳定运行。

在智能客服系统中，数据库可用于存储用户信息、聊天记录、知识库内容以及业务数据，支持 FAQ 管理、历史记录查询以及数据分析等功能。这不仅提高了系统的运行效率，也增强了数据的可管理性和安全性。

（5）在本系统中的应用

在本文设计的智能客服平台中，数据库技术被用于实现系统数据的统一管理。系统通过设计合理的数据表结构，对企业知识、用户信息以及交互记录进行存储；在用户提问时，通过数据库查询获取相关信息，并结合业务逻辑进行处理后返回结果。

通过引入数据库技术，系统能够实现数据的高效存储与快速访问，提升整体运行

性能。同时，也为后续的数据分析与功能扩展提供了良好的基础，从而更好地满足实际客服场景的需求[13]。

2.4 Spring Boot+Vue

在智能客服系统的设计与实现过程中，系统开发技术的选择对系统性能、稳定性以及用户体验具有重要影响。随着 Web 应用的发展，前后端分离架构逐渐成为主流开发模式。本文采用 Spring Boot 作为后端开发框架，Vue 作为前端开发框架，通过两者的结合构建一个高效、可扩展的智能客服系统[15]。

（1）Spring Boot 后端框架

Spring Boot 是基于 Spring 生态体系的快速开发框架，其通过“约定优于配置”的设计理念，大幅减少了传统 Spring 项目中繁琐的配置过程。开发者只需进行少量配置，即可快速构建一个完整的 Web 应用系统，从而显著提高开发效率。

在系统架构中，Spring Boot 主要负责处理核心业务逻辑，其功能包括接收前端请求、处理用户输入、调用大语言模型接口以及返回处理结果等。同时，Spring Boot 提供了完善的 Web 开发支持，可以通过 Spring MVC 实现 RESTful 风格的接口设计，使前后端通信更加规范化[16]。

此外，Spring Boot 具有良好的扩展能力，可以方便地集成多种技术组件。例如，通过整合 MyBatis 或 JPA 可以实现对数据库的高效访问，用于存储用户信息、对话记录以及系统日志等数据；通过引入 Spring Security 可以实现用户身份认证与权限控制，提升系统安全性；通过整合日志框架（如 Logback）可以实现系统运行状态的监控与记录。这些特性使 Spring Boot 能够满足智能客服系统在实际应用中的多样化需求[15]。

（2）Vue 前端框架

Vue 是一种轻量级且高效的前端开发框架，其核心思想是通过组件化方式构建用户界面。开发者可以将页面划分为多个独立的功能组件，例如聊天窗口组件、消息列表组件以及输入框组件等，从而提高代码的复用性和可维护性。

在智能客服系统中，Vue 主要负责实现用户交互界面。通过其响应式数据绑定机制，当后端返回新的对话内容时，前端界面可以自动更新，无需手动刷新页面，从而提供流畅的用户体验。此外，Vue 支持虚拟 DOM 机制，可以在数据变化时高效更新页面，减少不必要的渲染开销[15]。

在实际开发中，Vue 通常结合 Axios 实现与后端接口的数据通信。用户在界面中

输入问题后，前端通过 Axios 向后端发送请求，并在接收到返回结果后动态更新聊天内容。同时，Vue 还支持路由管理（Vue Router）和状态管理（Vuex），可以进一步提升系统结构的清晰性和复杂场景下的数据管理能力。

（3）前后端分离架构

本系统采用前后端分离的架构模式，即将用户界面与业务逻辑解耦，分别由前端和后端独立实现。前端主要负责界面展示与用户交互，后端则负责业务处理与数据管理，两者通过 HTTP 协议进行通信，并以 JSON 格式作为数据交换标准。

这种架构具有多方面优势。首先，前后端可以并行开发，提高开发效率；其次，系统结构更加清晰，便于后期维护与升级；再次，当系统需要扩展功能时，可以在不影响另一端的情况下进行独立修改。此外，前后端分离还便于系统部署，例如可以将前端部署在 CDN 上以提高访问速度，而后端部署在服务器上以保证数据安全[16]。

在智能客服系统中，该架构能够很好地支持高并发访问场景，同时提升系统的响应速度和稳定性。

（4）系统交互流程

在实际运行过程中，系统的交互流程主要包括用户输入、请求处理和结果返回三个阶段。首先，用户在前端界面输入问题，并通过输入框提交请求；随后，前端通过 HTTP 请求（通常为 POST 请求）将数据发送至后端接口。

后端接收到请求后，对用户输入进行解析，并根据业务逻辑调用相应服务，例如调用大语言模型进行文本生成，或通过检索模块获取知识库内容。在完成处理后，后端将生成的回答以 JSON 格式返回给前端。最后，前端接收到响应数据后，将其渲染到聊天界面中，实现人机对话交互[15]。

在这一过程中，通常采用异步请求机制（Ajax），可以避免页面阻塞，提高用户操作的流畅性。同时，通过接口规范设计，可以确保前后端数据传输的一致性和可靠性。

（5）系统优势与应用价值

采用 Spring Boot 与 Vue 相结合的开发模式，使系统在功能实现和性能表现方面具有明显优势。Spring Boot 提供稳定、高效的后端支持，能够处理复杂业务逻辑并保证系统运行的可靠性；Vue 则通过高效的前端渲染机制，为用户提供良好的交互体验。

此外，前后端分离架构使系统具有较强的扩展能力。例如，在后续开发中，可以

方便地增加用户管理模块、数据分析模块或多渠道接入功能（如移动端应用）。这种灵活性使系统能够适应不同业务场景的需求[16]。

在智能客服系统中，该技术方案不仅能够支持实时对话，还可以满足高并发访问需求，从而提升整体服务效率和用户满意度。

3 系统需求分析

3.1 系统概述

在基本电商功能的基础上，引入了智能客服功能，使平台不仅可以完成传统的商品交易，还能够通过对话的方式为用户提供服务。

系统整体仍采用 Spring Boot 与 Vue 进行开发，主要用于完成用户管理、商品管理以及页面展示等基础功能。在此基础上，加入了基于 LangChain 实现的 AI 客服模块，使用户可以通过输入自然语言完成部分操作，例如查询商品、处理订单等，从而提升系统的使用体验。

因此，可以理解为在传统电商结构上，增加了一个具备一定理解能力和执行能力的智能交互模块，使用户的操作方式更加灵活，不再完全依赖页面点击。

3.2 功能需求分析

结合实现情况，功能可以分为三部分：用户端功能、管理端功能以及 AI 客服功能。

（1）用户端功能需求

该部分功能模块主要用于支撑电商平台的基本使用流程，同时为 AI 模块提供稳定的数据基础。具体来说，它不仅承担用户下单、商品浏览、订单查询、支付结算等核心业务操作的支撑，还负责采集、整理和存储用户行为数据、商品信息以及交易记录，为智能客服系统的 AI 模块提供结构化和非结构化数据输入。

在电商场景中，用户咨询的问题多样且涉及不同业务环节，例如商品库存、价格、促销信息、物流状态、退换货政策等。该功能模块通过标准化的服务接口，将各类业务数据进行统一管理和调度，使 AI 模块能够快速、准确地获取所需信息，实现对用户问题的即时响应。此外，模块通过记录用户操作日志和交互历史，为模型提供上下文信息和行为参考，增强多轮对话的连贯性和回答的精准性。

通过上述设计，该功能模块不仅保证了电商平台日常业务的顺畅运行，还为智能客服平台提供了可靠的数据基础，使 AI 模块能够基于实时、完整的业务数据进行知识检索、生成式问答和任务执行，从而提升用户体验、提高服务效率，并支持复杂场景下的智能化决策。

用户用例如图 3-1 所示：

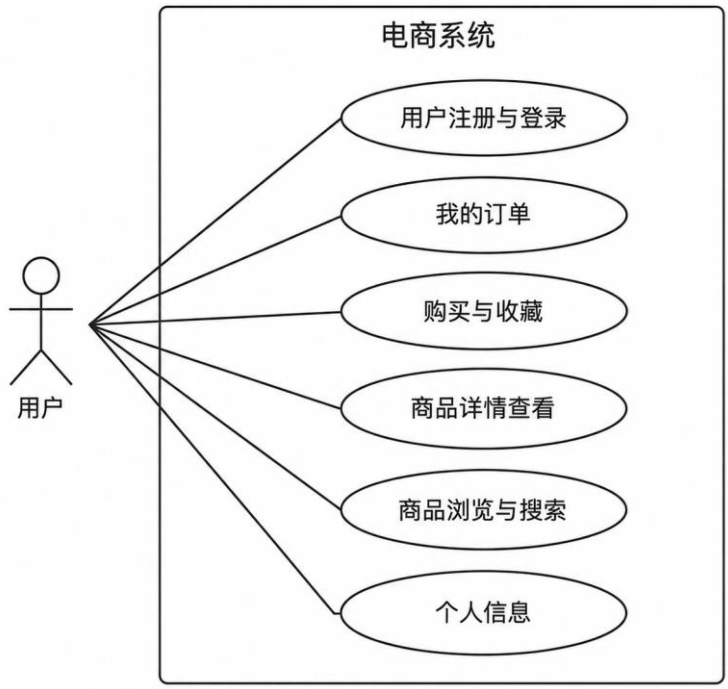


图3-1　用户用例图

用户端主要用于满足基本的购物需求，功能相对常规，包括：

用户注册与登录：用户可以通过账号密码进行注册和登录，系统通过 Token 进行身份验证。

商品浏览与搜索：用户可以按照分类浏览商品，也可以通过关键词进行搜索。

商品详情查看：在商品详情页中，可以查看商品图片、价格、库存等信息。

购买与收藏：用户可以将感兴趣的商品加入收藏，方便后续查看，以及购买商品。

我的订单：用户可以查看自己的订单，包括未支付订单和已支付订单。

（2）管理端功能需求

后台主要用于对系统数据进行管理，功能包括：

用户管理：管理员可以查看用户信息，并进行简单的管理操作。

商品管理：支持商品的新增、修改、上下架以及库存调整。

分类管理：对商品类别进行维护，方便前端展示。

订单管理：可以查看各个用户的订单信息，并进行简单的管理操作。

收藏信息：可以对每个用户的收藏信息进行管理。

平台管理：可以对电商平台的公告等模块进行管理。

其主要作用是保证系统数据的正常维护和更新。管理员的用例如图 3-2 所示：

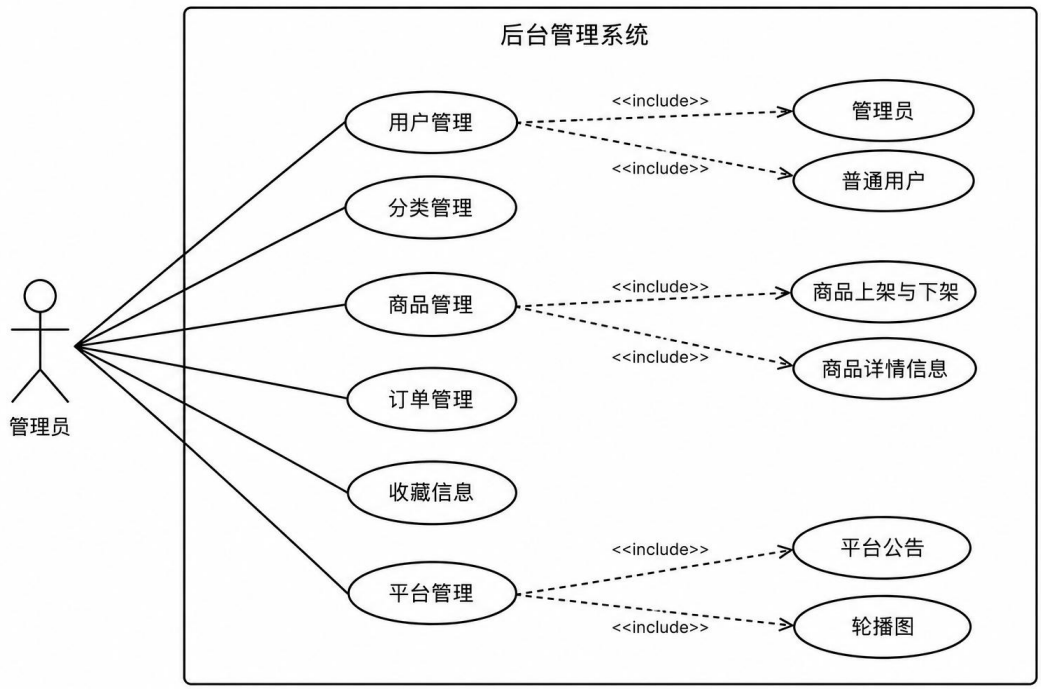


图3-2　管理员用例图

（3）AI 智能客服分析

本系统的重点在于 AI 客服模块的设计与实现。该模块基于 LangChain 框架，通过将大语言模型与后端接口结合，使其不仅可以回答问题，还可以完成实际操作。

主要功能包括：

对话交互：用户可以通过输入自然语言与系统进行交流，无需固定格式。

上下文理解：在多轮对话中，系统可以结合前面的内容理解用户意图，而不是每次都单独处理。

业务操作能力：

系统中封装了多个与电商相关的功能接口，并以工具（Tools）的形式提供给 AI 调用，例如：商品购买，订单取消，商品销量查询，库存查询，商品推荐

当用户提出需求时，系统会根据内容选择合适的功能进行处理，而不是简单返回文本结果。

结果反馈：在完成操作后，系统会将结果以自然语言形式返回给用户，使交互更

加友好。

回答范围控制：通过设置提示信息，对 AI 的回答范围进行限制，使其主要围绕电商相关内容展开。

相比传统客服系统，该模块更接近真实的人工服务过程，不仅可以“回答”，还可以“帮用户做事”。AI 客服的用例如图 3-3 所示：

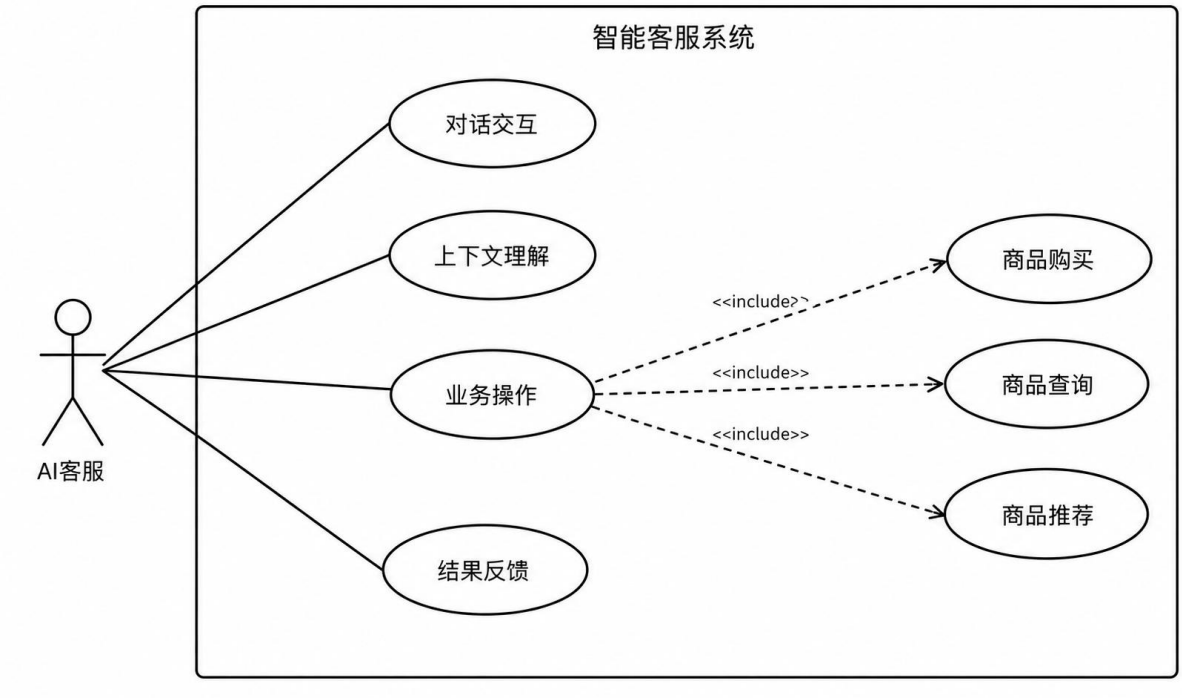


图3-3　AI客服用例图

3.3 非功能需求分析

（1）系统安全性需求

认证与授权：系统必须采用前后端分离的鉴权方案。后端通过 TokenUtils 生成加密 Token，前端在每次请求的 Header 中携带该令牌，未授权请求应统一拦截并重定向至 Login.vue。

接口防篡改：针对 collect、user 等敏感接口，需验证操作者身份，防止跨站请求伪造（CSRF）。

（2）系统性能需求

数据库优化：对于 goods 表的查询，需通过合理设置索引（如商品名称、类型 ID）来优化检索效率，其数学表达式可描述为查询复杂度降低。

并发库存一致性：在模拟用户下单或修改库存时，需通过事务管理机制确保数据

的一致性，防止超卖现象发生。

（3）可维护性与易用性需求

组件化设计：前端需遵循 Vue 的组件化规范，将公共部分提取为 Footer.vue 等组件，降低代码耦合度。

错误处理机制：后端应建立全局异常拦截器（GlobalException），针对 404、500 等错误返回统一格式的 Result 对象，便于前端进行友好提示。

（4）可行性分析

技术可行性：SpringBoot 2.7+ 搭配 JDK 17 提供了极强的性能支撑；Vue 2/3 在社区有大量的 UI 库（如 ElementUI）支持，能快速构建美观的后台界面。

经济可行性：采用 MySQL 等开源软件，开发成本低；集成第三方 AI 接口按量计费，适合中小型系统起步。

运行可行性：系统环境依赖简单，支持 Node.js 环境与 Maven 快速部署，具备在各类云服务器上稳定运行的条件。

4 系统设计

4.1 系统架构设计

本系统在整体结构上采用前后端分离的开发模式，前端使用 Vue 框架实现页面展示、用户交互以及数据的动态渲染，后端基于 Spring Boot 搭建业务服务，负责提供数据接口、处理业务逻辑以及完成与数据库之间的数据交互。前后端之间通过 HTTP 接口进行通信，使页面展示逻辑与后端业务逻辑相互解耦，从而提高系统的可维护性、扩展性和开发效率。

在传统电商系统功能的基础上，系统进一步引入了基于 LangChain 构建的智能客服模块，使系统不仅能够完成商品浏览、商品搜索、订单管理、收藏管理等常规电商业务，还能够通过自然语言交互的方式为用户提供智能化服务。用户可以通过聊天窗口向智能客服提出商品咨询、购买建议、订单相关问题等需求，智能客服模块能够对用户输入内容进行理解，并结合系统已有的商品、订单等业务数据，生成较为准确的回复，从而提升用户使用体验。

系统总体架构如图 4-1 所示：

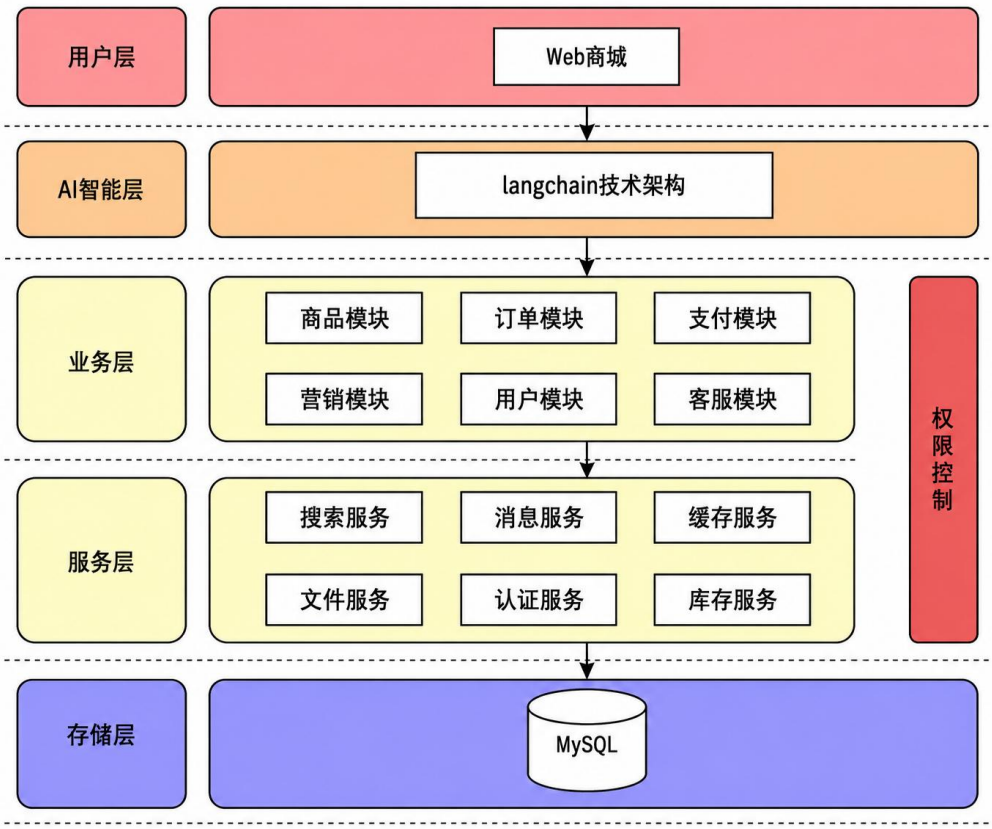


图4-1　系统总体架构图

其执行流程如下：

（1）用户在聊天窗口输入购买意图。

（2）后端接收消息后，将当前问题与最近几轮对话拼接为完整提示。

（3）大模型分析用户意图，并调用商品查询或购买工具。

（4）工具从数据库中读取商品、用户余额、库存等真实数据。

（5）若库存充足且余额足够，则自动生成待支付订单。

（6）若库存不足或余额不足，则返回明确失败原因及差额信息。

（7）最终由模型将业务结果组织为自然、简洁的客服回复返回前端。

4.2 系统功能模块设计

围绕“智能化电商平台”这一核心目标，在传统电商系统功能基础上，融合基于 LangChain 的大语言模型能力，对系统功能模块进行了重新设计与扩展。整体上，系统采用模块化设计思想，将平台划分为用户管理模块、商品管理模块、订单管理模块、智能客服模块等多个子模块，各模块之间通过接口进行解耦与协作，从而保证系统的可扩展性与可维护性。系统功能模块图如图 4-2 所示：

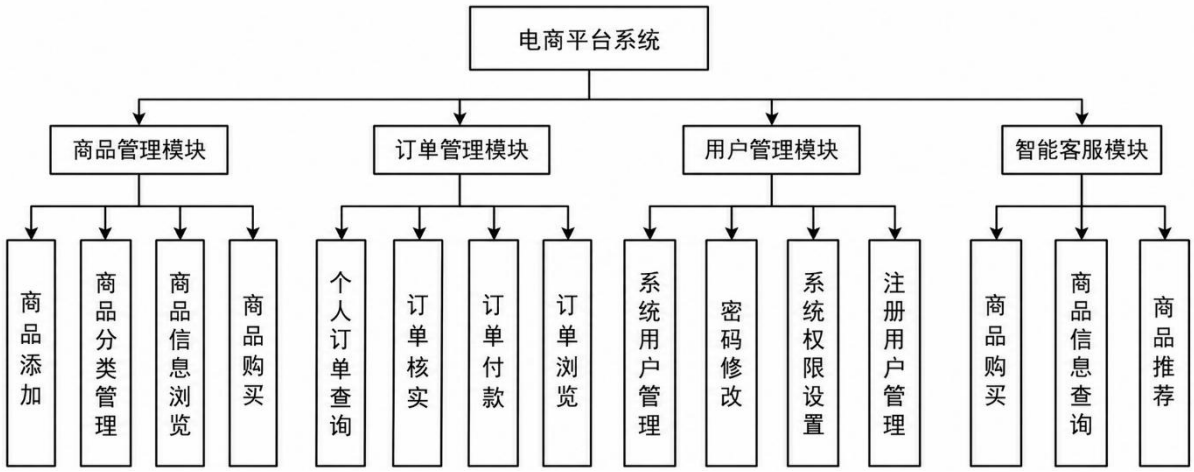


图4-2　系统功能模块图

4.2.1 用户管理模块

用户模块主要用于平台的基础管理与维护，包含系统用户管理、注册用户管理、密码修改以及系统权限设置等功能。用户模块流程图如图 4-3 所示：

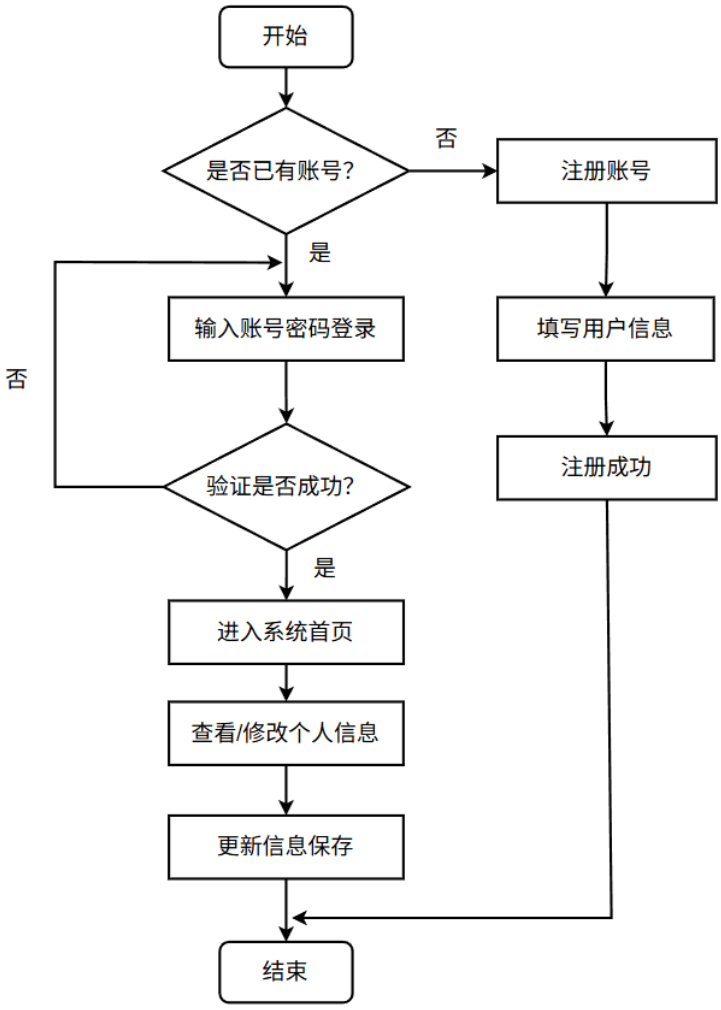


图4-3　用户管理模块流程图

4.2.2 订单管理模块

订单模块是电商系统中实现交易闭环的核心模块，也是整个智能客服平台业务支撑的重要基础。其主要职责是管理订单的生成、确认、支付、更新及查询等功能，确保用户从选购商品到完成交易的整个流程顺畅、高效。

具体来说，当用户在平台上完成商品选择后，系统会根据购物车信息生成对应订单，并为订单分配唯一标识以便跟踪管理。用户可对生成的订单进行核实，检查商品内容、数量、价格、优惠信息、税费及运费等相关费用，确保订单信息的准确性与完整性。系统在订单确认后，将允许用户进行支付操作，支持多种支付方式，并在接收到支付信息后及时更新订单状态，从“待支付”变更为“已支付”或“交易完成”，保证交易流程的透明化与实时性。

订单管理流程图如图 4-4 所示：

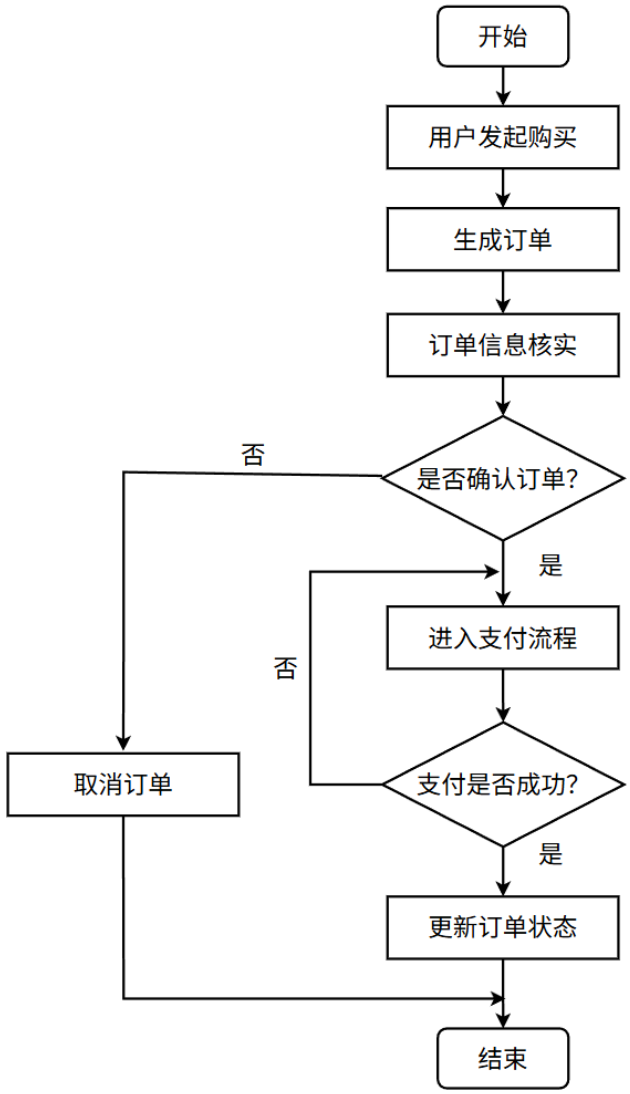


图4-4　订单管理模块流程图

4.2.3 商品管理模块

系统支持管理员对商品进行添加操作，将商品名称、价格、库存及描述等信息录入系统，并通过商品分类管理对商品进行合理划分，从而提高用户查找商品的效率。在前端展示方面，用户可以浏览商品的详细信息，包括价格、图片及相关描述，从而全面了解商品属性。在此基础上，系统提供商品购买功能，用户在浏览商品后可直接发起购买请求，系统将根据用户选择生成对应订单，并与订单模块进行衔接，实现完整的购物流程。商品管理如图 4-5 所示：

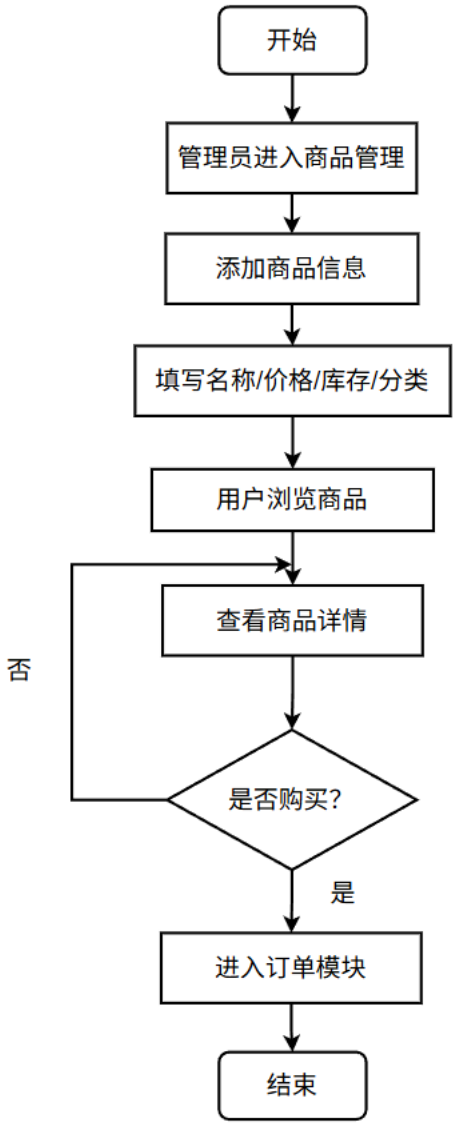


图4-5　商品管理模块流程图

4.2.4 智能客服模块

为了让 AI 具备实际操作能力，系统将电商相关的核心业务封装为多个工具（Tools），并注册到 LangChain 中供模型调用。每个工具对应一个具体功能接口，AI

在判断用户意图后，会自动选择合适的工具执行。

Tools 工具如表 4-1 所示：

表4-1 Tools工具

Tool 名称	功能描述
buyGoodsTool	实现商品购买
cancelOrderToo	实现订单取消
querySalesTool	查询商品销量
queryStockTool	查询库存
recommendGoodsTool	商品推荐

每个工具的执行过程可以抽象为一个函数调用过程,如公式 4-1：

$$Result = f(Tool, Input) \quad (4-1)$$

其中，Tool 表示具体功能模块，Input 为模型解析后的参数信息，Result 为执行结果。通过这种方式，可以将复杂的业务逻辑简化为统一的调用形式，降低系统耦合度。

在实际运行过程中，Agent 会经历如下步骤，Agent 执行流程如图 4-6 所示：

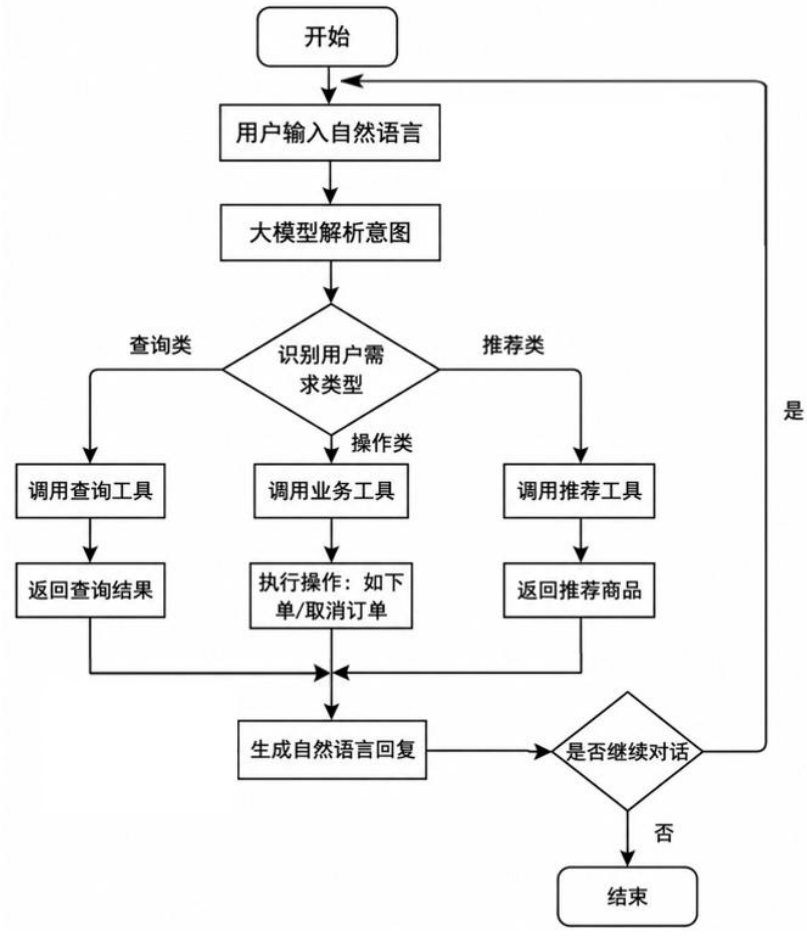


图4-6　Agent执行流程图

在实际运行过程中，Agent 通常遵循“思考—行动—观察”的循环机制。首先，系统对用户输入进行语义解析，明确用户意图；其次，根据分析结果选择合适的工具并执行相应操作；然后，获取工具返回的结果，并将其作为新的信息输入模型；最后，结合当前结果生成最终回答或进入下一轮决策过程。

需要注意的是，Agent 的决策过程并非基于简单的关键词匹配，而是依赖于大语言模型对自然语言的深层语义理解能力。例如，当用户输入“帮我看看这个商品卖得怎么样”时，系统并不会仅依据“商品”或“看看”等关键词进行处理，而是能够理解其真实意图为“查询商品销量或销售情况”，从而调用相应的数据查询工具。这种基于语义理解的决策方式显著提升了系统的智能化水平和任务处理的准确性。

此外，在部分复杂场景下，用户的需求可能包含多个步骤，例如“先帮我查一下库存，如果有货就下单”。此时，Agent 可以通过多次调用工具逐步完成任务，这也是 LangChain 在实际应用中的一个重要优势。

Prompt 工程（Prompt Engineering）是围绕大语言模型（LLM）的一项核心技术，指的是通过系统性设计和优化输入提示（Prompt），来引导模型生成更准确、稳定且符合预期的输出。随着 GPT、Claude 等模型能力增强，用户与模型的交互方式从“简单提问”逐渐演变为“结构化指令设计”，Prompt 工程也因此成为连接人类意图与模型能力的关键桥梁。

从原理上看，大模型本质是基于概率预测的文本生成系统，它并不真正“理解”问题，而是根据输入上下文预测最可能的输出。因此，提示词的表达方式会直接影响模型的理解路径和输出空间。模糊的提示会导致结果发散，而清晰、具体、带有约束的提示则能显著提升输出质量。这也是 Prompt 工程的核心目标：减少不确定性，引导生成方向，提高结果可控性。

一个高质量的 Prompt 通常由几个关键要素构成。首先是“角色设定”（Role），即为模型指定一个身份，例如“资深数据分析师”或“法律顾问”，这有助于模型在特定语境下组织语言和知识。其次是“任务描述”（Task），需要明确说明希望模型完成什么工作，例如“撰写报告”“解释概念”或“生成代码”。第三是“背景信息”（Context），提供必要的上下文，如目标用户、使用场景或相关数据，这能帮助模型更准确地理解问题。最后是“输出约束”（Constraints），包括格式（如表格、JSON）、风格（正式或口语）、长度限制等，这一步可以让输出更结构化、更易用。

在实际应用中，Prompt 工程还发展出一系列常见技巧。比如“Few-shot Prompting”（少样本提示），通过给出几个输入输出示例，让模型学习模式并进行类比生成；“Chain-of-Thought”（思维链提示），通过引导模型逐步推理（如“请一步一步分析”），可以显著提升复杂问题（尤其是数学和逻辑问题）的正确率；“分解任务”（Task Decomposition）则是将复杂问题拆分为多个简单步骤，降低模型一次性处理的难度。此外，还有“自一致性”（Self-consistency）、“反思提示”（Reflection）等进阶方法，用于提高推理稳定性和答案质量。

本系统的 Prompt 模板如图 4-7 所示：

```
package com.example.springboot.service.ai;

import ...

public interface CustomerSupportAssistant {  5 个用法

    @SystemMessage("""  2 个用法
            你是商城 AI 客服，请像真人客服一样自然交流，不要机械堆砌字段。
            你的职责：
            1. 根据用户需求推荐商品。
            2. 查询商品价格、库存、销量、上架时间等信息。
            3. 查询当前登录用户的订单、收藏和账户余额信息。
            4. 当用户明确要购买商品时，优先调用购买工具生成订单；购买成功时说明订单已生成且状态为待付款。
            5. 收藏中的商品也可以购买，用户明确提到“收藏里的”“我收藏的”时优先使用收藏购买工具。
            6. 如果余额不足导致购买失败，必须明确告知失败原因、商品金额、当前余额和还差多少，不要编造成功结果。
            7. 用户明确要求取消订单时，可以调用工具帮他取消。
            8. 如果用户说“取消我的手机订单”“查一下刚才那个订单”这类话，先结合最近几轮上下文理解，不要立刻重复追问。
            9. 涉及准确商品数据、订单数据、收藏数据、余额数据时，必须先调用工具，不能编造。
            10. 回复使用简体中文，语气自然、礼貌、简洁，尽量控制在 2 到 4 行。
            """)
    String chat(@UserMessage String message);
}
```

图4-7　Prompt模板

Prompt 工程的应用场景非常广泛。在内容创作中，它可以用于生成文章、广告文案或邮件；在编程领域，可以帮助生成代码、解释错误或进行调试；在数据分析中，可以辅助总结数据、生成报告；在教育场景中，可以作为个性化辅导工具；在企业中，还常用于构建 AI Agent 或自动化流程。可以说，只要涉及人机语言交互的地方，Prompt 工程都有价值。

从更高层次来看，Prompt 工程不仅是一种技巧，更是一种“人与 AI 协作的思维方式”。它要求用户具备清晰表达需求的能力，将模糊想法转化为结构化指令。随着工具的发展，一些复杂 Prompt 甚至被模块化、模板化，形成可复用的“提示系统”，

在团队或产品中反复使用。

当然，Prompt 工程也存在局限性。例如，不同模型对同一 Prompt 的响应可能不同，提示效果具有一定不稳定性；随着模型能力提升，一些简单任务对 Prompt 的依赖在降低；此外，过于复杂的 Prompt 也可能增加维护成本。因此，在实践中需要在“简单有效”和“精细控制”之间取得平衡。

4.3 数据库设计

为了保证系统数据存储的规范性、一致性与可扩展性，采用关系型数据库对各类业务数据进行统一管理。在数据库设计过程中，首先结合系统功能需求进行分析，明确系统中涉及的核心数据对象，并基于实体-关系（E-R）模型对数据结构进行抽象建模。在此基础上，将概念模型进一步转化为逻辑结构，形成具体的数据表设计，从而支撑系统各项功能的实现。系统总体的 ER 图，如图 4-8 所示：

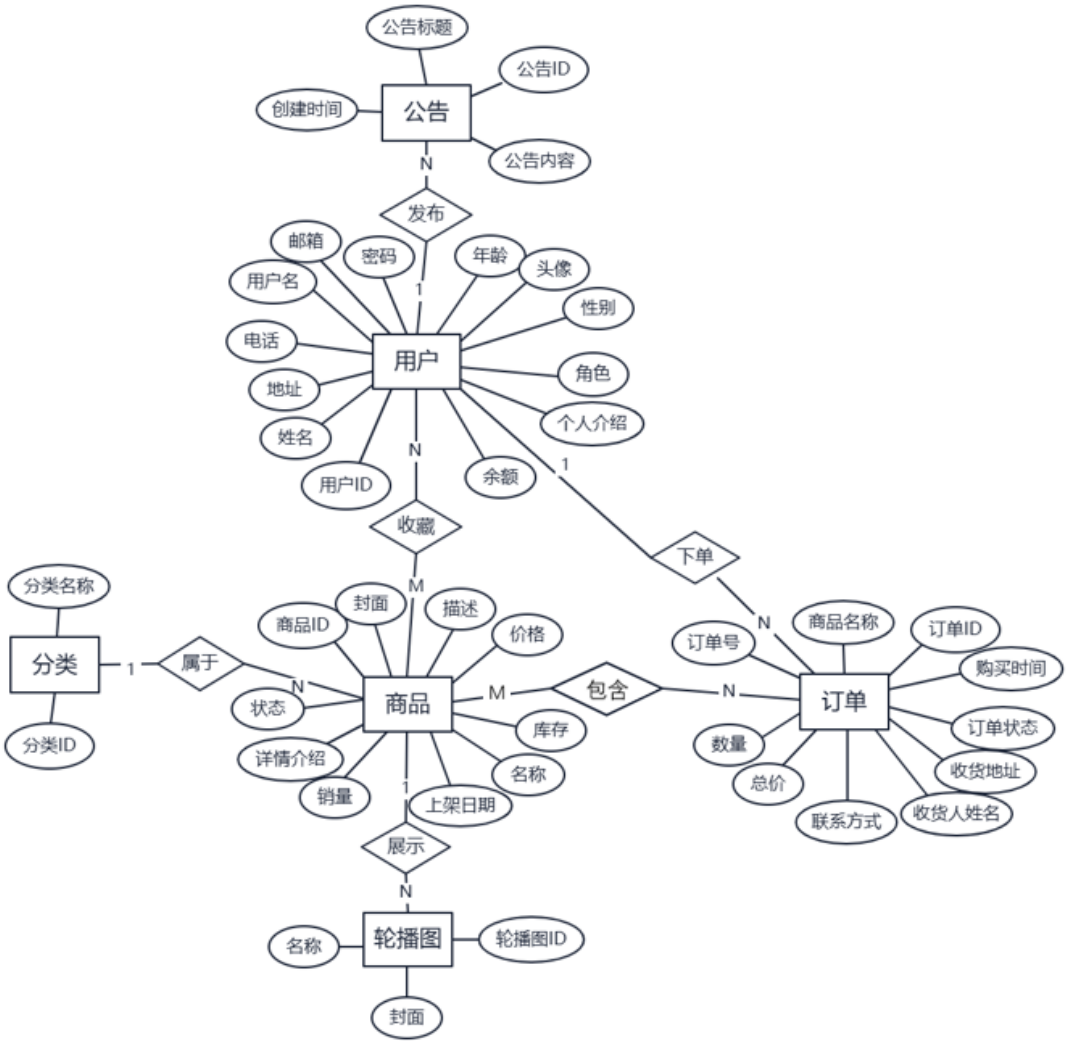


图4-8　系统E-R图

在业务关系方面，分类与商品之间构成一对多关系，即一个分类下可以包含多个商品；用户与订单之间同样为一对多关系，一个用户可以生成多个订单记录；商品与订单之间存在关联关系，用于记录具体的购买行为。此外，用户与商品之间还存在收藏关系，该关系属于多对多关系，因此通过建立中间表的方式进行实现，从而保证数据结构的规范性。公告实体用于系统信息发布，与用户之间构成一对多关系，即一个用户可以发布多条公告信息。

（1）轮播图表

轮播图表用于存储系统首页展示的轮播图片信息，主要包括轮播图的名称、封面图片以及所关联的商品 ID 等内容。该表的核心作用是实现前端页面的广告展示或推荐展示功能，一条轮播图记录通常对应一个商品，用户点击轮播图后可以跳转到对应的商品详情页面，因此轮播图表与商品表之间存在多对一的关联关系。具体的表结构如表 4-2 所示：

表 4-2 轮播图表

字段名	数据类型	数据长度	字段描述	字段说明
Carouser_id	int	4	ID	主键，非空
name	varchar	255	名称	
cover	varchar	255	封面	
goods_id	int	4	商品 ID	

（2）商品表

商品表是系统中的核心数据表之一，主要用于存储平台中所有商品的基本信息，是商品展示、商品管理以及交易流程的重要数据基础。商品表中包含商品名称、商品描述、详细介绍、商品价格、库存数量、商品封面、上架时间、商品状态以及所属分类等字段，用于完整记录商品的各项属性信息。每个商品都具有唯一的主键标识，通过该标识可以准确区分不同商品，保证商品数据的唯一性和完整性，同时也方便其他数据表对商品信息进行引用。在系统实际运行过程中，用户浏览商品列表、查看商品详情、搜索商品信息以及下单购买商品时，都需要依赖商品表中的相关数据，例如商品名称和封面用于前台页面展示，商品描述和详细介绍用于商品详情介绍，价格和库存则直接影响用户是否能够正常购买。此外，商品表还与多个业务数据表存在关联关系，每个商品都归属于某一个分类，因此商品表与分类表之间存在联系，便于系统按

照分类对商品进行管理、筛选和展示；商品可以被用户收藏，因此商品表与收藏表之间存在关联，用于记录用户对商品的收藏行为；商品还可以被用户加入订单，成为订单中的购买对象，因此商品表与订单表之间也存在联系，用于记录订单中所包含的商品信息。具体的表结构如表 4-3 所示：

表 4-3 商品表

字段名	数据类型	数据长度	字段描述	字段说明
goods_id	int	4	ID	主键，非空
name	varchar	255	名称	
descr	varchar	255	描述	
content	text	255	详情介绍	
cover	varchar	255	封面	
price	double	10	价格	
store	int	4	库存	
user_id	int	4	添加人	
date	varchar	255	上架日期	
type_id	int	4	分类 ID	
state	varchar	255	状态	
sales	int	4	销量	

（3）收藏表

收藏表用于记录用户对商品的收藏行为，主要包含用户 ID、商品 ID 以及收藏时间等信息。通过该表可以实现用户收藏商品的功能，每一条记录表示某个用户收藏了某个商品，因此收藏表与用户表和商品表之间均存在多对一关系，用于体现用户与商品之间的多对多关系。此外，在实际应用中，可以为收藏表建立联合唯一索引（如用户 ID 和商品 ID）能提升查询效率。具体的表结构如表 4-4 所示：

表 4-4 收藏表

字段名	数据类型	数据长度	字段描述	字段说明
collect_id	int	4	ID	主键，非空
user_id	int	4	用户 ID	
goods_id	int	4	商品 ID	
time	varchar	255	收藏时间	

（4）用户表

用户表用于存储系统中所有用户的基本信息，包括用户名、密码、姓名、电话、邮箱、地址以及头像等内容。每个用户通过唯一的主键 ID 进行标识，并可以进行下单、收藏商品等操作，因此用户表与订单表和收藏表存在关联关系，是系统中行为发起的主体。具体的表结构如表 4-5 所示：

表 4-5 用户表

字段名	数据类型	数据长度	字段描述	字段说明
user_id	int	4	ID	主键，非空
username	varchar	255	用户名	
password	varchar	255	密码	
name	varchar	255	姓名	
phone	varchar	255	电话	
email	varchar	255	邮箱	
address	varchar	255	地址	
avatar	varchar	255	头像	
sex	varchar	255	性别	
age	int	4	年龄	
infos	varchar	255	个人介绍	
role	varchar	255	角色	

（5）订单表

订单表用于记录用户在系统中的购买行为信息，是电商系统中非常重要的业务数据表之一。该表主要用于保存订单相关的核心数据，包括订单号、商品名称、商品价格或订单总价、用户联系方式、用户名、用户姓名、下单时间以及订单状态等内容。通过订单表，系统可以完整地记录用户从购买商品到生成订单的整个交易过程，为后续的订单查询、订单管理、数据统计以及售后处理提供数据支持。

在实际业务中，每一条订单记录通常对应用户的一次购买行为。当用户选择商品并提交订单后，系统会自动生成唯一的订单号，并将该订单的商品信息、用户信息、价格信息以及联系方式等内容保存到订单表中。订单号作为订单记录的重要标识，能够用于区分不同订单，方便用户和管理员对订单进行查询、管理和跟踪。

具体的表结构如表 4-6 所示：

表 4-6 订单表

字段名	数据类型	数据长度	字段描述	字段说明
order_id	int	4	ID	主键，非空
name	varchar	255	商品名称	
order_no	varchar	255	订单号	
goods_id	int	4	商品 ID	
nums	int	4	数量	
user_name	varchar	255	姓名	
state	varchar	255	订单状态	
price	double	10	总价	

（6）分类表

分类表用于对商品进行类别划分，主要存储分类的名称以及主键 ID 等信息。每个商品都归属于某一个分类，而一个分类下可以包含多个商品，因此分类表与商品表之间是一对多关系，该表的存在有助于实现商品的分类展示与管理功能。具体的表结构如表 4-7 所示：

表 4-7 分类表

字段名	数据类型	数据长度	字段描述	字段说明
category_id	int	4	ID	主键，非空
name	varchar	255	商品分类名称	

（7）公告表

公告表用于记录和管理电商平台中的公告信息，是平台管理功能的重要组成部分。系统通过公告表对平台公告进行统一存储，使管理员能够在后台对公告内容进行新增、修改、删除和查询操作，同时也方便普通用户在前台页面查看平台发布的通知信息。公告内容可以包括系统维护通知、平台活动说明、商品促销信息、使用规则提醒以及其他需要向用户展示的重要信息。

通过公告表的设计，系统能够对平台公告信息进行统一管理。管理员可以在后台发布新的公告内容，也可以根据实际需要对已有公告进行维护；普通用户则可以在前台页面查看平台发布的公告信息，及时了解系统通知、商品活动或平台规则变化等内容。因此，公告表不仅承担着信息存储的作用，也增强了平台与用户之间的信息传递

能力。公告表如表 4-8 所示：

表 4-8 公告表

字段名	数据类型	数据长度	字段描述	字段说明
notice_id	int	4	ID	主键，非空
name	varchar	255	商品分类名称	
content	text	255	公告内容	
time	varchar	255	创建时间	
user_id	int	4	创建人	

5 系统功能实现

5.1 用户模块

5.1.1 用户注册与登录

用户注册与登录功能用于完成用户身份的创建与验证，是系统运行的基础。当用户进行注册时，在前端输入用户名和密码并提交请求，系统接收到请求后会先对用户名进行唯一性校验，即判断数据库中是否已存在该用户名。如果用户名已经存在，则返回提示信息；用户登录界面如图 5-1 所示：

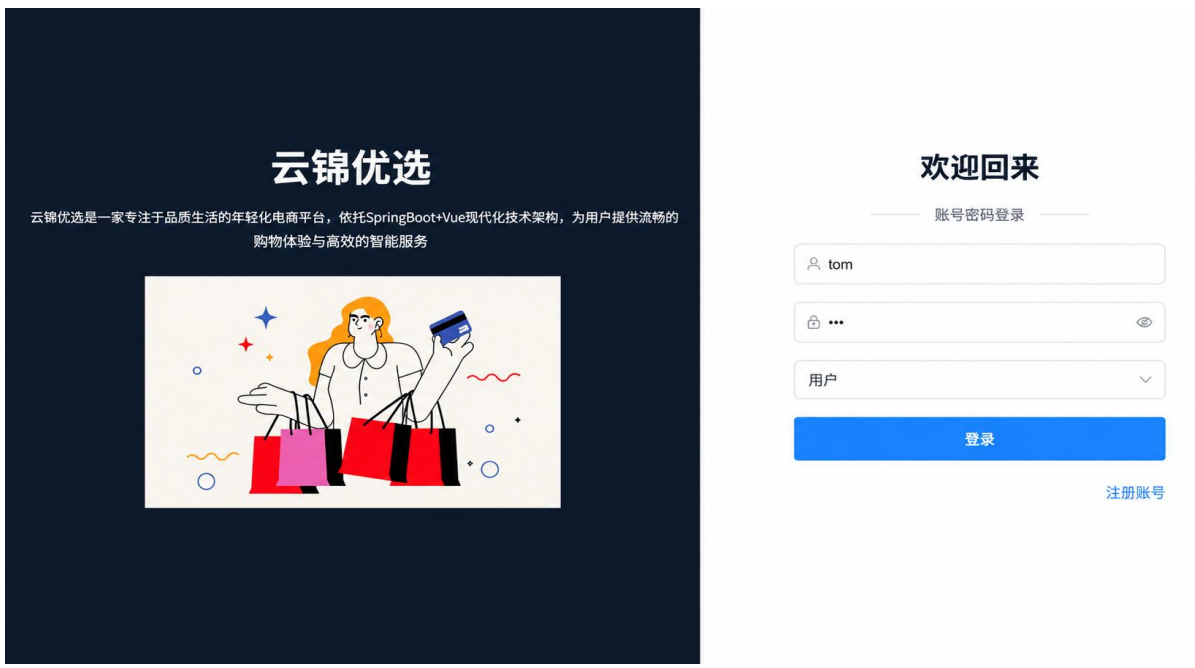


图5-1　用户登录界面

用户注册界面如图 5-2 所示：



图5-2　用户注册界面

这段代码展示了用户注册与登录的基本流程，作为系统的公共 Web 接口控制器，提供了三个主要接口：一个用于测试连通性的 hello 接口，以及用户登录和注册功能。登录接口接收 JSON 格式的用户数据，会对用户名和密码进行非空校验，然后交由 userService 处理。

关键代码如图 5-3 所示：

```
@PostMapping("/login")
public Result login(@RequestBody User user) {
    if (StrUtil.isBlank(user.getUsername()) || StrUtil.isBlank(user.getPassword())) {
        return Result.error( msg: "数据输入不合法");
    }
    user = userService.login(user);
    return Result.success(user);
}

/**
 * 注册
 */
@PostMapping("/register")
public Result register(@RequestBody User user) {
    if (StrUtil.isBlank(user.getUsername()) || StrUtil.isBlank(user.getPassword())) {
        return Result.error( msg: "数据输入不合法");
    }
    if (user.getUsername().length() > 10 || user.getPassword().length() > 20) {
        return Result.error( msg: "数据输入不合法");
    }
    if (StrUtil.isBlank(user.getRole())) {
        user.setRole("USER");
    }
    user = userService.register(user);
    return Result.success(user);
}
```

图5-3　用户登录注册关键代码

采用统一的 Result 封装返回结果，通过 success 和 error 方法标准化响应格式。系

统集成了 JWT 拦截器用于登录鉴权，配置了跨域支持以适应前后端分离的部署方式，同时引入了 LangChain4j 的配置。

5.1.2 用户购买

用户购买模块用于实现用户下单购买商品的功能。当用户在系统中选择商品并发起购买请求时，前端会将用户信息、商品信息以及购买数量等数据提交到后端。系统接收到请求后，会首先校验用户是否存在以及商品是否有效，同时判断商品库存是否满足购买需求。在确认信息无误后，系统会根据商品价格和购买数量计算订单相关信息，并生成对应的订单记录，将订单数据保存到数据库中。最后返回订单创建成功的结果，从而完成一次完整的购买流程，该过程实现了用户与商品之间的业务关联。

用户购买界面如图 5-4 所示：

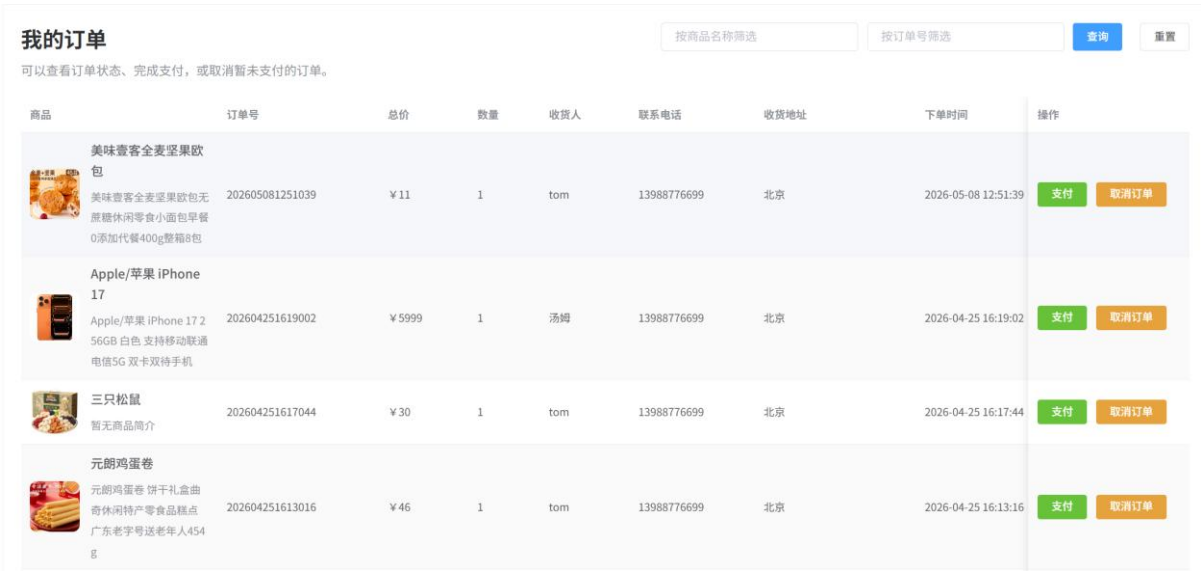


图5-4　用户购买界面

这段代码展示了用户购买的基本流程，通过@RestController 和@RequestMapping("/orders")注解定义了订单模块的基础路由路径。该类注入了IOrdersService 服务接口，实现了订单的增删改查全套操作：add 方法使用@PostMapping 接收 POST 请求并调用 save 保存订单数据，update 方法通过@PutMapping 处理 PUT 请求完成订单更新 delet 方法借助@DeleteMapping 接收订单 ID 参数执行删除操作，selectAll 方法则以 @GetMapping 暴露 GET 接口返回全部订单列表。关键代码如图 5-5 所示：

```java
@RequestMapping("/orders")
public class OrdersController {

    @Autowired
    private IOrdersService ordersService;

    @PostMapping("/add")
    public Result add(@RequestBody Orders orders) {
        ordersService.save(orders);
        return Result.success();
    }

    @PutMapping("/update")
    public Result update(@RequestBody Orders orders) {
        ordersService.update(orders);
        return Result.success();
    }

    @DeleteMapping("/delete")
    public Result delete(@RequestParam Integer id) {
        ordersService.remove(id);
        return Result.success();
    }

    @GetMapping("/selectAll")
    public Result selectAll() { return Result.success(ordersService.selectAll()); }

    @GetMapping("/selectById")
    public Result selectById(@RequestParam Integer id) { return Result.success(ordersService.selectById(id)); }
```

图5-5　用户购买关键代码

5.2 商品模块

5.2.1 商品分类管理

商品分类管理用于对系统中的商品进行分类管理，从而提升商品组织和查询效率。当管理员添加商品分类时，需要输入分类名称，系统接收请求后会创建对应的分类数据并保存到数据库中。对于分类查询功能，系统会从数据库中读取所有分类信息，并将结果返回给前端进行展示，使用户可以按照分类浏览商品。该模块的核心在于对分类数据的维护以及分类信息的统一管理，为商品展示和后续功能提供基础支持。商品分类界面如图 5-6 所示：

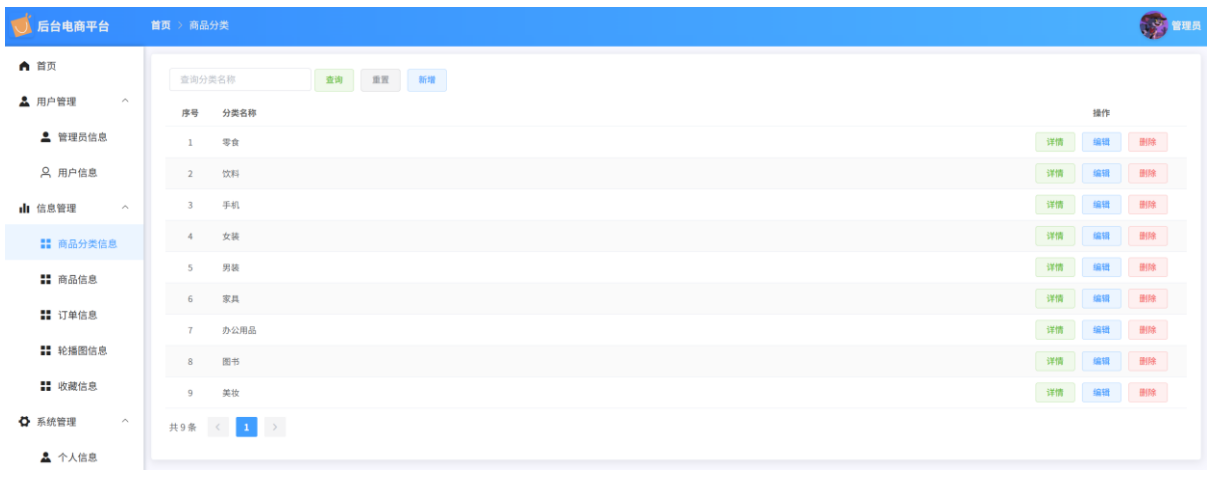


图5-6　商品分类管理界面

这段代码展示了商品分类的实现流程，通@RestController 和@RequestMapping 注解定义了商品分类模块的基础路由，内部注入了 ITypeService 服务接口，实现了分类信息的增删改查全套操作：add 方法采用@PostMapping 接收 POST 请求并调用 ave 保存分类数据，update 方法通过 @PutMapping 处理 PUT 请求完成分类更新，delete 方法借助@DeleteMapping 接收 ID 参数执行删除 selectAll 和 selectById 则分别通过@RequestMapping 暴露查询接口返回全部列表和单条记录。关键代码如图 5-7 所示：

```
public class TypeController {
    private ITypeService  typeService;

    @PostMapping("/add")
    public Result add(@RequestBody Type type) {
        typeService.save(type);
        return Result.success();
    }

    @PutMapping("/update")
    public Result update(@RequestBody Type type) {
        typeService.update(type);
        return Result.success();
    }

    @DeleteMapping("/delete")
    public Result delete(@RequestParam Integer id) {
        typeService.remove(id);
        return Result.success();
    }

    @RequestMapping("/selectAll")
    public Result selectAll() { return Result.success(typeService.selectAll()); }

    @RequestMapping("/selectById")
    public Result selectById(@RequestParam Integer id) { return Result.success(typeService.selectById(id)); }

    @RequestMapping("/selectPage")
    public Result selectPage(@RequestParam(defaultValue = "") String name,
                             @RequestParam Integer pageNum,
                             @RequestParam Integer pageSize) {
        return Result.success(typeService.selectPage(pageNum, pageSize, name));
```

图5-7　商品分类关键代码

5.2.2 用户订单管理

用户订单管理功能主要用于对系统中的订单数据进行统一管理和维护。系统可以根据需求查询所有订单信息，或者根据用户标识筛选出对应用户的订单记录，从而实现订单的查看功能。同时，系统也支持根据订单编号删除指定订单，这一操作通常用于后台管理。在实际应用中，该模块还可以扩展订单状态管理功能，例如对订单设置“待支付”“已完成”等状态，以实现更加完整的订单生命周期管理。用户订单管理界面如图 5-8 所示：



图5-8　用户订单管理界面

5.3 智能客服模块

智能客服模块是本系统的核心创新点，基于 LangChain 框架构建，结合大语言模型与工具调用机制，实现从传统问答式客服向任务型智能助手的转变。

关键代码如图 5-9 所示：

```
private String toDetailSummary(Goods goods) {  1 个用法
    return String.format("%s，售价%s元，库存%s件，销量%s件，上架时间%s。商品简介：%s。",
            defaultText(goods.getName()),
            formatAmount(safeDouble(goods.getPrice())),
            defaultNumber(goods.getStore()),
            defaultNumber(goods.getSales()),
            defaultText(goods.getDate()),
            defaultText(goods.getDescr()));
}

private String toOrderSummary(Orders order) {  4 个用法
    return String.format("订单号%s，商品%s，总金额%s元，数量%s件，状态%s，下单时间%s。",
            defaultText(order.getOrderNo()),
            defaultText(order.getName()),
            formatAmount(safeDouble(order.getPrice())),
            defaultNumber(order.getNums()),
            defaultText(order.getState()),
            defaultText(order.getTime()));
}

private String toCollectSummary(Collect collect) {  2 个用法
    Goods goods = collect.getGoods();
    if (goods == null) {
        return "你收藏过一件商品，但商品信息暂时读取失败。";
    }
    return String.format("你收藏了%s，当前价格%s元，库存%s件。",
            defaultText(goods.getName()),
            formatAmount(safeDouble(goods.getPrice())),
            defaultNumber(goods.getStore()));
```

图5-9　智能客服关键代码

这段代码展示了智能客服模块的基本功能 GoodsAiTools 类内部注入了 IUserService 服务接口，封装了面向自然语言交互的商品查询、订单管理和用户操作等功能：listGoods 方法支持查询商城商品列表以进行商品推荐，queryGoodsByName 方法可按商品名称查询包含价格、库存、销量、上架时间和简介在内的详细信息，hotSalesGoods 方法用于查询热销商品列表，recommendGoodsByPriceRange 方法支持按价格区间筛选商品以适配预算推荐场景。在订单管理方面，myOrders 方法可查询当前登录用户的全部订单列表，queryMyOrderByOrderNo 方法支持按订单号查询特定订单详情，queryMyOrdersByGoodsName 方法则能按商品关键词检索相关订单。此外，系统还提供了 myCollects 方法查询用户收藏列表、queryMyAccountBalance 方法查询账户余额，以及 buyGoodsByName 和 buyCollectsByGoodsName 方法实现基于商品名称或收藏商品的智能购买功能，这些购买操作会自动生成待付款订单并校验账户余额。

5.3.1 商品推荐

商品推荐功能通过 AI 技术分析用户输入的需求，从而为用户提供合适的商品建议。当用户输入类似“推荐一款手机”的需求时，系统会将该输入传递给 AI 模块进行处理，AI 会识别用户意图并提取关键需求信息，然后结合系统中的商品数据进行匹配，筛选出符合条件的商品并生成推荐结果返回给用户。该过程可以通过简单的关键词匹配实现，也可以结合更复杂的推荐算法或大模型能力，从而提升推荐的准确性和个性化程度。智能客服商品推荐如图 5-10 所示：



图 5-10　智能客服商品推荐

5.3.2 商品查询

商品查询功能用于帮助用户通过自然语言快速查找所需商品。当用户输入查询请求时，例如询问某类商品或具体商品名称，系统会利用 AI 对输入内容进行解析，识别出查询意图并提取相关关键词，然后在商品数据库中进行匹配搜索，获取符合条件的商品信息，并将查询结果返回给用户。如果系统未能匹配到相关商品，则会返回提示信息，引导用户调整查询内容，从而提升交互体验和查询效率。商品查询如图 5-11 所示：



图 5-11　智能客服商品查询

5.3.3 商品购买

商品购买功能通过 AI 对话的方式引导用户完成购买流程。当用户通过自然语言表达购买需求时，系统首先通过 AI 识别其购买意图，并从用户输入中提取商品名称或相关特征信息，然后在商品数据库中查找对应商品，并向用户展示商品信息以进行确认。在用户确认购买后，系统会调用订单模块生成订单并完成购买流程。如果用户表达不清晰，AI 会进一步引导用户补充必要信息，例如商品型号或数量，从而确保购买流程能够顺利完成。该功能实现了智能交互与业务逻辑的结合，使用户可以通过对话完成购物操作。商品购买如图 5-12 所示：



图 5-12　智能客服商品购买

6 总结与展望

6.1 全文总结

本文围绕电商系统的智能化方向，设计并实现了一个融合传统业务与智能交互能力的电商平台。在整体实现上，系统仍采用 Spring Boot 与 Vue 构建基础框架，用于完成用户管理、商品管理及页面展示等常规功能。但与一般电商系统不同的是，本文将研究重点放在 AI 客服模块的设计与实现上，尝试通过引入 LangChain，使系统具备一定的理解与执行能力。

在实际开发过程中，完成了从需求分析、系统设计到具体功能实现的完整流程。其中，最核心的工作是基于 LangChain 构建智能客服模块，并将其与电商业务进行结合。通过对业务接口进行封装，设计了多个可供模型调用的工具（Tools），使系统不仅能够进行对话，还可以根据用户需求完成具体操作。

本课题的主要工作可以概括为以下几个方面：

（1）完成基础系统搭建

基于前后端分离模式，实现了电商平台的基本功能，包括商品浏览、用户管理、订单处理等，为后续 AI 功能提供了稳定的数据与接口支持。

（2）构建基于 LangChain 的智能客服体系

通过引入 LangChain 框架，将大语言模型与实际业务进行连接，使 AI 不仅用于文本生成，还可以参与系统功能调用。这种方式突破了传统客服仅限于问答的局限。

（3）实现多工具（Tools）协同调用机制

针对电商场景，设计并实现了多个功能工具，包括商品购买、订单取消、商品销量查询、库存查询以及商品推荐等。AI 可以根据用户输入自动选择合适工具，从而完成对应操作，使交互更加接近真实人工服务过程。

（4）实现“对话驱动操作”的交互模式

用户无需按照固定流程点击页面，而是可以通过自然语言直接表达需求，例如下单或查询信息。系统通过意图识别与工具调用，将用户输入转化为具体业务行为，提升了整体使用体验。

（5）提升系统整体灵活性

通过将 AI 模块与传统业务逻辑解耦，系统在后续扩展时只需新增工具即可增加功能，避免对原有结构进行大规模修改。

通过本项目的开发实践，不仅加深了对 Java 后端开发和前端技术的理解，更重要的是，对大语言模型在实际系统中的应用方式有了较为直观的认识，尤其是在“模型如何参与业务流程”这一问题上获得了一定经验。

6.2 不足与展望

尽管系统已经实现了预期功能，并能够完成基本运行，但在实际使用和进一步扩展方面，仍存在一些有待完善的地方：

（1）AI 与业务数据结合仍可加强

当前 AI 虽然可以调用工具完成操作，但在部分场景下，对实时数据的利用还不够深入。例如库存和销量等信息仍依赖接口返回，未来可以进一步优化，使 AI 能够更高效地获取并整合数据，从而提升回答的准确性与实时性。

（2）复杂任务处理能力有待提升

目前系统主要支持单一任务调用，对于多步骤或组合任务（如“比较商品后再下单”）的处理能力还不够完善。后续可以通过优化 Agent 策略，使其具备更强的任务拆解与执行能力。

（3）对话体验仍有提升空间

在连续对话过程中，虽然已经实现上下文关联，但在部分复杂语境下仍可能出现理解偏差。未来可以通过改进上下文管理机制或引入更合理的记忆策略，提高对话的连贯性。

（4）系统性能与扩展能力

当前系统主要在单机环境下运行，对于高并发场景考虑较少。后续可以结合缓存机制或异步处理方式，提高系统在大规模访问下的稳定性。

（5）功能拓展方向

在现有基础上，可以进一步增加更多工具类型，例如智能比价、自动推荐组合商品等，使 AI 在电商场景中的作用更加丰富。同时，也可以探索接入知识库，使客服具备更强的专业回答能力。

总体来看，本课题在传统电商系统的基础上，尝试引入 LangChain 实现智能客服，并通过工具调用机制让 AI 参与实际业务处理。这种模式为电商系统的智能化提供了一种较为直观的实现思路。后续随着相关技术的发展，AI 在系统中的角色也有望从辅助工具逐步转变为更加核心的组成部分。

参考文献

[1] 姜嘉伟.基于 Langchain-LLMs 框架的智能问答系统的设计与实现[D].延边大学硕士学位论文,2024:81-89.

[2] Auffarth B, Kuligin L. Generative AI with LangChain: Build Production-Ready LLM Applications and Advanced Agents Using Python, LangChain, and LangGraph[M]. Birmingham: Packt Publishing Limited, 2025-05-23: 65-69.

[3] 张宁.基于 LangChain 的 Python 知识问答系统的设计与实现[J].现代信息科技,2026,10(05):66-70.

[4] 陈文丽,张洪波,靳德政.基于 LangChain 的检索增强生成知识库问答系统设计与实现[J].计算机时代,2026,(02):51-56.

[5] 尹强.基于大模型和知识图谱的问答助手构建——以医学教育为例[J].电脑知识与技术,2026,22(02):25-28.

[6] 李晨阳,蔡冬,凌立,等.基于外挂专业知识库的 AI 大模型搭建方案与其在智能制造中的应用[J].现代信息科技,2025,9(16):194-198.

[7] 罗萍,刘哲宇.电商平台落地 AI 技术的五大方向[J].中国眼镜科技杂志,2026,(03):16-18.

[8] 曾庆彬,张嘉鑫,吴开心.电商平台推荐系统的设计与实现[J].福建电脑,2026,42(01):61-67.

[9] GUO W ,WANG B ,HAN D , et al. Tourism Demand Forecasting Based on Sentiment Analysis and EST-Net Prediction Model[J].Journal of Systems Science and Information,2025,13(05):726-750.

[10] 郭健洋.基于大语言模型的空管专业多模态文档语义分析研究[J].成都信息工程大学学报,2025,40(06):806-811.

[11] 石善忠.基于 LangChain+Deepseek 的智慧燃气 GIS 领域问答系统研究与实现[J].现代信息科技,2025,9(22):12-16.

[12] 刘安骞.基于 LangChain 与 DeepSeek 的 JavaScript 教学辅助智能体的研究[J].现代信息科技,2025,9(07):76-82.

[13] 李长英,管玲玲.基于深度学习的数据库查询优化方法研究[J].电脑编程技巧与维护,2026,(03):104-106.

[14] 陈晓男.数据库设计与应用[M].北京:中国铁道出版社,2024,08:233.

[15] 杨开振.深入浅出 Spring Boot 3.x[M].北京:人民邮电出版社,2024:404-408.

[16] 贾冲冲,马文光,王金龙,等.基于 SpringBoot 的采购信息管理系统的设计与实现[J].电脑编程技巧与维护,2026,(02):103-107.

学士学位论文（设计）评审表

<table>
<tr><td>学院</td><td>人工智能与计算机学院</td><td>姓名</td><td>司洪宇</td><td>导师姓名</td><td>孙玉霞</td></tr>
<tr><td>专业</td><td>人工智能</td><td>学号</td><td>2022116060628</td><td>导师职称</td><td>副教授</td></tr>
<tr><td>论文题目</td><td colspan="5">基于 LangChain 的智能客服平台的设计与实现</td></tr>
<tr><td>论文主要内容简介</td><td colspan="5">本文围绕“基于 LangChain 的智能客服平台的设计与实现”展开研究，针对传统电商系统中客服交互方式单一、用户操作流程复杂以及智能化程度不足等问题，设计并实现了一个融合大语言模型技术的智能电商平台。系统采用 Spring Boot 与 Vue 前后端分离架构，实现了用户管理、商品管理、订单管理及分类管理等基础电商功能，并在此基础上引入基于 LangChain 框架的 AI 智能客服模块，通过自然语言处理技术实现用户与系统之间的智能交互。在系统设计中，重点结合大语言模型、LangChain 框架及 MySQL 数据库技术，构建了以 Agent 机制为核心的智能业务调度体系，将商品购买、订单取消、库存查询、销量查询及商品推荐等功能封装为 Tools 工具，使智能客服不仅具备问答能力，还能够直接执行具体业务操作，实现从传统问答式客服向任务型智能助手的转变。在系统实现过程中，用户可通过自然语言完成商品查询、推荐与购买等操作，系统支持多轮对话与上下文理解，显著提升了交互体验与操作便捷性。</td></tr>
<tr><td>论文评语</td><td colspan="5">导师（签名）：　　年　　月　　日</td></tr>
<tr><td>总评成绩</td><td colspan="5"></td></tr>
<tr><td colspan="6">学院答辩委员会主席(签名或盖章)：_____　　学院盖章：</td></tr>
</table>

注：本表将装订在论文正文后面，务必认真填写。